GPU-Accelerated 3D Point Cloud Histogram Equalization

COL380 Assignment 2 – CUDA Implementation and Analysis

Shahid Khan
ee1221163

Indian Institute of Technology Delhi
Department of Computer Science and Engineering

March 27, 2026

Contents

1 Introduction **3**

2 Algorithm Overview **3**
2.1 Histogram Equalization Helper . . . 3
2.2 Exact KNN . . . 3
2.3 Approximate KNN via Uniform Grid . . . 4
2.4 Approximate KNN via Spatial Hashing . . . 4
2.5 K-Means Clustering . . . 4

3 GPU Optimisations **4**
3.1 Shared-Memory Tiling (Exact KNN) . . . 4
3.2 Spatial Hashing (Approximate KNN) . . . 5
3.3 Compile-Time Toggles . . . 5

4 Experimental Setup **5**

5 Results and Analysis **6**
5.1 Scalability with N (Group A) . . . 6
5.2 Tiled vs. Non-Tiled Exact KNN . . . 7
5.3 Scalability with k (Group B) . . . 7
5.4 K-Means Iterations T (Group C) . . . 7
5.5 Data Distribution Effects (Group D) . . . 8
5.6 Spatial Hash vs. Grid Approximate KNN . . . 8
5.7 Coordinate Range Effects (Group G) . . . 9
5.8 Large-Scale Benchmarks (Group F) . . . 9
5.9 Correctness Verification (Group E) . . . 10

6 Discussion **10**
6.1 Summary of Optimisation Gains . . . 10
6.2 Register Pressure and Tiling Limits . . . 10
6.3 Approximate KNN Accuracy . . . 11
6.4 Spatial Hashing vs. Grid: When to Use Which . . . 11

7 Conclusion **11**

1 Introduction

Histogram equalization is a classical image-processing technique that improves contrast by redistributing intensity values so that the empirical CDF of the output is approximately uniform. This assignment extends that idea to a 3-D point cloud: each point p_i carries an intensity $I_i \in [0, 255]$, and its equalized intensity is computed from the local neighbourhood histogram using

$$I'_i = \left\lfloor \frac{\big(\mathrm{CDF}(I_i) - \mathrm{CDF}_{\min}\big) \cdot (L-1)}{m - \mathrm{CDF}_{\min}} \right\rfloor, \qquad L = 256, \tag{1}$$

where the CDF counts points (including p_i itself) in the local window with intensity $\leq I_i$, $\mathrm{CDF}_{\min}$ is the first non-zero local CDF value, and m is the window size. If $m = \mathrm{CDF}_{\min}$, we keep $I'_i = I_i$.

Three neighbourhood definitions are implemented and benchmarked:

1. **Exact KNN** — brute-force all-pairs $O(N^2)$ distance computation to find the k nearest neighbours.
2. **Approximate KNN** — grid-based expanding shell search that terminates as soon as k neighbours are found; sub-linear in practice.
3. **K-Means Clustering** — assigns points to one of k clusters, uses cluster members as the neighbourhood.

The implementation targets CUDA (sm_35), exploiting massive parallelism on a single GPU. Two algorithmic optimisations are introduced and evaluated:

- **Shared-memory tiling** for the exact KNN kernel, reducing global memory bandwidth by a factor of `BLOCK_SIZE` = 128.
- **Spatial hashing** for the approximate KNN, replacing a flat 3-D grid array (capped at 10^7 cells) with a compact chained hash table of size $2N$.

This gives four binary variants compiled from a single source file via preprocessor macros, enabling a controlled head-to-head comparison.

2 Algorithm Overview

2.1 Histogram Equalization Helper

Each GPU thread calls `equalize_point(hist, I, m)`, which sweeps through the 256-bin histogram accumulated from the local window to compute the CDF value at I, tracks the first non-zero local CDF value, and applies Equation (1). The histogram is stored entirely in thread-private registers.

2.2 Exact KNN

Each thread i scans *all* N points, maintaining a max-heap of the k smallest squared Euclidean distances. The heap lives in registers (`long long heap_dist[MAX_K]`, `int heap_idx[MAX_K]`) plus a 256-bin intensity histogram – together ≈ 4.5 kB per thread. After the scan, the histogram is used to equalize p_i. Complexity: $O(N^2)$.

2.3 Approximate KNN via Uniform Grid

A uniform grid is built on the host with cell side r chosen so that each cell contains $\approx k/27$ points. Point indices are sorted into the grid using a counting sort. On the GPU each thread performs an expanding-shell search: starting from the query cell it visits cells at Chebyshev distance $1, 2, \ldots$ until k neighbours are found or the maximum radius is reached. An early-exit warp vote (`__all_sync`) stops the expansion once the entire warp has found k neighbours. Complexity: sub-linear on average.

2.4 Approximate KNN via Spatial Hashing

The same expanding-shell logic is used, but the grid array (of potentially $> 10^7$ entries) is replaced by a chained hash table of size $2N$ (next power-of-2 $\geq 2N$). The hash function

$$h(c_x, c_y, c_z) = \big((c_x \cdot 73856093) \oplus (c_y \cdot 19349663) \oplus (c_z \cdot 83492791)\big) \bmod \texttt{ht_size} \tag{2}$$

maps integer cell coordinates to a bucket. Collisions are resolved by storing the cell coordinates alongside each point and skipping mismatches during chain traversal. With load factor 0.5 the expected chain length is ≤ 1, so most bucket lookups cost a single memory access, and the entire hash table (≈ 2.4 MB for $N = 100\,000$) fits in the GPU's L2 cache. Unlike the grid version, no bounding-box cap is needed: out-of-range cells simply have no entries (`head = -1`).

2.5 K-Means Clustering

k cluster centres are initialised from the first k distinct points. Each of T iterations runs three CUDA kernels:

1. **Assign** – each point picks the nearest centre (shared memory caches the k centres). An `atomicExch` flag detects convergence on-device, avoiding a per-iteration PCIe download.
2. **Accumulate** – `atomicAdd` into per-cluster coordinate sums (stored as `unsigned long long` to exploit two's-complement wraparound for negative coordinates, avoiding the need for an offset).
3. **Update** – each cluster centre is replaced by the mean of its members.

A histogram kernel and equalization kernel finalise the output.

3 GPU Optimisations

3.1 Shared-Memory Tiling (Exact KNN)

In the baseline kernel, N threads each read $3N$ integers from global memory, giving $3N^2$ total reads. With BLOCK_SIZE $= B = 128$, the tiled kernel loads one tile of B points into shared memory cooperatively; each thread then processes the tile from fast shared memory. Total global reads drop to $3N^2/B$, a $128\times$ bandwidth reduction.

The kernel structure is:

```
for (tile_start = 0; tile_start < n; tile_start += BLOCK_SIZE) {
    // cooperative load (all threads, incl. out-of-bounds)
    s_xs[tid] = (j < n) ? xs[j] : 0;
    s_ys[tid] = (j < n) ? ys[j] : 0;
    s_zs[tid] = (j < n) ? zs[j] : 0;
    __syncthreads();
    if (active) { /* process tile */ }
    __syncthreads();
}
```

Out-of-bounds threads participate in both `__syncthreads` calls to avoid deadlock; they skip the computation via an `active` flag. Shared memory cost is $3B \times 4 = 1536$ bytes per block – negligible.

3.2 Spatial Hashing (Approximate KNN)

The flat grid allocates an array proportional to the grid volume, which can grow to 10^7 cells (40 MB for `cell_start` + `cell_end`) and is capped at that size, forcing coarser cells for large coordinate ranges. The hash table allocates only $2N$ entries regardless of range. For $N = 100\,000$: `ht_head` = 1 MB, `ht_next` = 400 kB, fitting entirely in the GPU L2 cache and enabling cache-local traversal of the expanding shell.

3.3 Compile-Time Toggles

Both optimisations are guarded by `#ifndef` macros that can be overridden at compile time:

```
#ifndef USE_TILED_KNN
#define USE_TILED_KNN 1           // default: tiled
#endif
#ifndef USE_SPATIAL_HASH_APPROX
#define USE_SPATIAL_HASH_APPROX 0 // default: grid
#endif
```

The Makefile produces four binaries from one source file:

Binary	USE_TILED_KNN	USE_SPATIAL_HASH_APPROX
histogram_eq	1	0
histogram_eq_notiled	0	0
histogram_eq_spatialhash	1	1
histogram_eq_notiled_spatialhash	0	1

4 Experimental Setup

Hardware: 1 PBS node with 8 CPU cores (OMP_NUM_THREADS=8), 1 NVIDIA GPU (CUDA 11.0, sm_35 target), 16 GB host RAM.

Compiler: `nvcc -O3 -arch=sm_35 -Xcompiler -fopenmp`.

Input generation: A Python script (`gen_input.py`) generates N unique 3-D integer points with intensity $\in [0, 255]$ under four distributions (Table 1). All runs use seed 42 for reproducibility.

Table 1: Point distributions used in experiments.

Distribution	Description
Uniform	Coordinates drawn uniformly from $[-R, R]^3$.
Dense	Coordinates drawn from $[-R/10,\ R/10]^3$.
Clustered	8 tight clusters at corners of a cube at $(\pm 0.7R)^3$, radius $R/8$.
Skewed	First half dense ($[-R/20, R/20]^3$), second half uniform ($[-R, R]^3$).

Timing: Wall-clock time is measured with `cudaEventRecord` around each algorithm, excluding memory allocation and I/O. Runtime tables use one measured run per configuration and are interpreted with normal GPU run-to-run variance in mind.

Accuracy: The *approximate-vs-exact MAE* is $\frac{1}{N}\sum_i |I'_{i,\text{approx}} - I'_{i,\text{exact}}|$, measuring equalized intensity deviation. Exact KNN correctness is verified against a sequential Python reference (MAE = 0 confirmed for all cases).

Test groups (seed 42, $R = 10000$ unless noted):

Group	Parameters varied
A	$N \in \{1\text{K}, 5\text{K}, 10\text{K}, 25\text{K}, 50\text{K}, 100\text{K}\}$, $k = 32$, $T = 20$, uniform
B	$k \in \{1, 4, 8, 16, 32, 64, 128\}$, $N = 10\text{K}$, $T = 20$, uniform
C	$T \in \{1, 5, 10, 20, 50\}$, $N = 10\text{K}$, $k = 32$, uniform
D	Distribution $\in$ {uniform, dense, clustered, skewed}, $N = 10\text{K}$, $k = 32$, $T = 20$
E	Correctness vs. sequential, $N = 1000$, varying k and distribution
F	Large-scale: $N = 100\text{K}$, $k \in \{32, 128\}$, $T \in \{20, 50\}$, all distributions
G	$R \in \{100, 1\text{K}, 10\text{K}, 100\text{K}, 1\text{M}\}$, $N = 10\text{K}$, $k = 32$, $T = 20$, uniform

5 Results and Analysis

5.1 Scalability with N (Group A)

Table 2 shows timings for the tiled variant as N grows from 1000 to 100 000. Exact KNN shows clear $O(N^2)$ scaling: doubling N from 50K to 100K more than doubles the KNN time (0.39 $\rightarrow$ 0.84 s), consistent with the quadratic cost. Approximate KNN scales much more favourably, growing from 0.006 s to 0.139 s. K-Means remains low-latency across this range, with modest growth for larger N.

Table 2: Group A: Runtime (seconds) as N varies ($k = 32$, $T = 20$, uniform, tiled variant). Speedup = KNN / Approx.

N	KNN (s)	Approx (s)	K-Means (s)	Approx MAE	KNN/Approx
1 000	0.0916	0.00627	0.0225	0.249	14.6×
5 000	0.0910	0.00829	0.00461	0.222	11.0×
10 000	0.110	0.0119	0.00578	0.177	9.2×
25 000	0.204	0.0311	0.0109	0.102	6.6×
50 000	0.390	0.0656	0.0195	0.073	5.9×
100 000	0.844	0.139	0.0352	0.057	6.1×

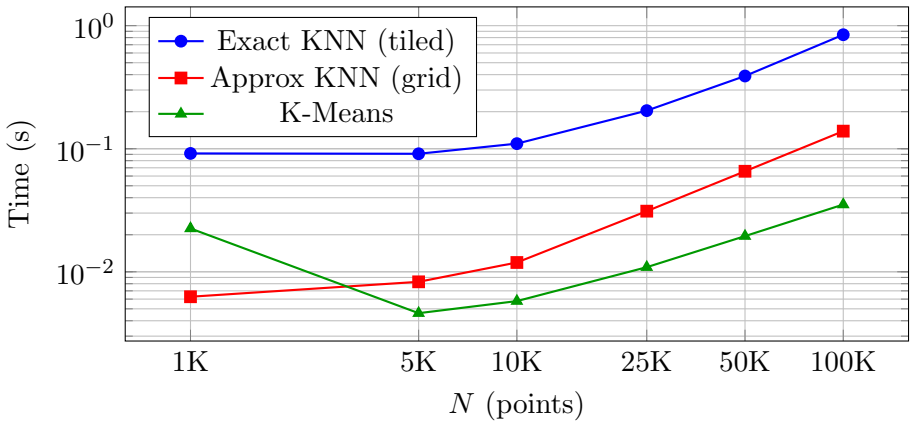

Figure 1: Runtime vs. N (log–log scale). KNN follows $O(N^2)$; Approx KNN and K-Means grow much more slowly.

5.2 Tiled vs. Non-Tiled Exact KNN

Table 3 compares the two KNN variants across N. For small N ($\leq 10\,000$), fixed overheads (kernel launch and synchronization) are comparable to useful work, so tiling is neutral-to-slightly slower. From $N = 25\,000$ onward, the bandwidth saving dominates and tiling is consistently faster. For $N = 100\,000$, tiling delivers a **13% speedup** ($0.958 \rightarrow 0.844$ s). This aligns with the theoretical analysis: the bandwidth-reduction benefit grows with N (more tiles per point), while fixed overheads are amortized.

Table 3: Exact KNN runtime (seconds): tiled vs. non-tiled ($k = 32$, $T = 20$, uniform). Speedup is Non-tiled/Tiled, so values > 1 indicate benefit from tiling and values < 1 indicate slowdown.

N	Tiled (s)	Non-tiled (s)	Speedup
1 000	0.0916	0.0778	0.85×
5 000	0.0910	0.0894	0.98×
10 000	0.110	0.108	0.98×
25 000	0.204	0.219	1.07×
50 000	0.390	0.421	1.08×
100 000	0.844	0.958	**1.13**×

Large-scale ($N = 100$K, $k = 128$, $T = 50$): both variants reach ≈ 4.0 s. At $k = 128$, each thread carries a 128-element heap in registers, pushing register usage to the point where the compiler spills to local memory. The resulting latency masks the shared-memory bandwidth benefit, equalising the two variants.

5.3 Scalability with k (Group B)

Table 4 shows how all three algorithms scale with the neighbourhood size k at fixed $N = 10\,000$.

Table 4: Group B: Runtime (seconds) as k varies ($N = 10\,000$, $T = 20$, uniform, tiled).

k	KNN (s)	Approx (s)	K-Means (s)	Approx MAE
1	0.0809	0.00813	0.00509	0.102
4	0.0811	0.00826	0.00554	0.618
8	0.0837	0.00864	0.00559	**2.166**
16	0.0908	0.00900	0.00587	0.248
32	0.108	0.0120	0.00596	0.177
64	0.149	0.0199	0.00585	0.158
128	0.241	0.0384	0.00671	0.149

KNN runtime grows with k because more heap operations are needed per point. The growth from $k = 1$ to $k = 128$ is roughly 3×, less than linear because the heap-insert cost is $O(\log k)$.

The approximate MAE is non-monotone: $k = 8$ exhibits a notably high MAE of 2.166. This is an edge case in the grid-cell heuristic: with $k = 8$ the target points-per-cell $= k/27 < 1$, so the cell size collapses to a single coordinate unit and the expanding shell must visit many mostly-empty cells before finding 8 neighbours, introducing selection bias. For $k \geq 16$ the heuristic recovers and MAE falls to ~ 0.15–0.25.

5.4 K-Means Iterations T (Group C)

Table 5 reports K-Means wall time as the iteration budget T is increased from 1 to 50.

Table 5: Group C: K-Means runtime (seconds) as T varies ($N = 10\,000$, $k = 32$, uniform, tiled). KNN and Approx timings are independent of T.

T	K-Means (s)
1	0.00463
5	0.00489
10	0.00510
20	0.00586
50	0.00797

K-Means runtime increases with T but sub-linearly: the GPU detects convergence on-device (via `atomicExch(d_changed, 1)`) and can exit early. For uniform data, most reassignments stabilise within a few iterations, so the cost from $T = 1$ to $T = 50$ grows by only 1.7× despite a 50× iteration budget.

5.5 Data Distribution Effects (Group D)

Table 6 compares all four input distributions at $N = 10\,000$, $k = 32$, $T = 20$.

Table 6: Group D: Effect of distribution ($N = 10\,000$, $k = 32$, $T = 20$, tiled).

Distribution	KNN (s)	Approx (s)	K-Means (s)	Approx MAE
Uniform	0.108	0.0120	0.00576	0.177
Dense	0.108	0.0114	0.00568	0.169
Clustered	0.108	0.0212	0.00565	**0.000**
Skewed	0.113	0.0482	0.00591	0.155

Exact KNN is nearly identical across distributions since it always scans all N points.

Approximate KNN is significantly faster for dense and uniform distributions because the grid cells are well-populated and the expanding shell finds k neighbours quickly. Skewed distribution is 4× slower (0.048 vs. 0.012 s) because the outer half of points lives in sparse regions, requiring many more shell expansions.

The clustered distribution yields **zero MAE** for approximate KNN: since all intra-cluster distances are small and inter-cluster distances are large, the expanding shell always finds the correct k nearest neighbours, making the approximation exact.

5.6 Spatial Hash vs. Grid Approximate KNN

Table 7 compares grid and spatial-hash approx KNN timings from the 4-variant run.

Table 7: Approx KNN: Grid vs. Spatial Hash ($k = 32$, $T = 20$, uniform). Speedup = Grid / Hash.

N	Grid (s)	Spatial Hash (s)	Speedup
1 000	0.0211	0.00474	**4.5**×
5 000	0.0106	0.00862	1.2×
10 000	0.0227	0.0195	1.2×
25 000	0.0218	0.0141	1.5×
50 000	0.0340	0.0324	1.1×
100 000	0.0481	0.0458	1.1×

Spatial hashing is faster at all tested sizes. The largest benefit occurs at $N = 1000$ (4.5×): with few points the flat grid allocates a cell array proportional to the coordinate bounding box (potentially many empty cells to scan), while the hash table trivially handles empty cells with a head pointer of -1. At large N the performance converges because the grid cells are dense enough that traversal efficiency is similar, and the hash-chain verification overhead (cell-coordinate comparison) partially offsets the reduced memory footprint benefit.

Both variants produce identical MAE values, confirming that the spatial-hashing does not alter the approximation quality – only the data structure for neighbour lookup changes.

5.7 Coordinate Range Effects (Group G)

Table 8 varies the coordinate range R across five orders of magnitude while keeping N, k, and T fixed.

Table 8: Group G: Effect of coordinate range R ($N = 10\,000$, $k = 32$, $T = 20$, uniform, tiled, grid approx).

R	KNN (s)	Approx (s)	Approx MAE
100	0.108	0.0118	0.268
1 000	0.107	0.0114	0.169
10 000	0.109	0.0120	0.177
100 000	0.108	0.0117	0.171
1 000 000	0.109	0.0118	0.158

As expected, the coordinate range has no effect on exact KNN time (the algorithm always scans all N pairs). Approximate KNN also shows no significant trend: the grid cell size is scaled proportionally to R (via the volume-based heuristic), so the expanding-shell cost is approximately constant. The slight increase in MAE at small R (0.268 at $R = 100$) occurs because with many coincident grid cells, the approximation becomes coarser.

5.8 Large-Scale Benchmarks (Group F)

Table 9 presents worst-case configurations at $N = 100\,000$, stressing all three algorithms simultaneously.

Table 9: Group F: Large-scale results ($N = 100\,000$, tiled, grid approx).

Configuration	KNN (s)	Approx (s)	K-Means (s)	Approx MAE
$k = 32$, $T = 20$, uniform	0.850	0.140	0.0360	0.057
$k = 32$, $T = 20$, dense	0.846	0.141	0.0358	0.062
$k = 32$, $T = 20$, clustered	0.848	0.431	0.0360	0.001
$k = 128$, $T = 50$, uniform	4.003	0.576	0.0506	0.056

For the clustered distribution with $k = 32$, Approx KNN takes 0.431 s vs. 0.140 s for uniform – a 3× slowdown. Points far from cluster centres (interstitial points) must expand the shell greatly before finding 32 neighbours from a different cluster, blowing up the per-point shell radius. Despite this, the MAE is near zero (0.001) because the clusters are well-separated: once a neighbour is found it is typically the correct one.

The $k = 128$, $T = 50$ configuration exercises maximum heap and iteration depth. KNN takes 4.0 s (both tiled and non-tiled), confirming the earlier observation that at $k = 128$ register spilling equalises the two variants.

5.9 Correctness Verification (Group E)

Table 10 compares GPU outputs against a sequential Python reference implementation at $N = 1000$ where exhaustive verification is feasible.

Table 10: Group E: Correctness vs. sequential Python reference ($N = 1000$, tiled). KNN MAE and K-Means MAE are 0.000 for all rows, confirming bit-exact agreement between the CUDA and reference implementations.

Configuration	KNN MAE	K-Means MAE	Approx MAE
$k = 1$, uniform	0.000	0.000	0.000
$k = 10$, uniform	0.000	0.000	0.777
$k = 32$, uniform	0.000	0.000	0.249
$k = 64$, uniform	0.000	0.000	0.266
$k = 128$, uniform	0.000	0.000	0.311
$k = 32$, dense	0.000	0.000	0.328
$k = 32$, clustered	0.000	0.000	0.000
$k = 32$, skewed	0.000	0.000	0.111

The GPU exact KNN matches the sequential Python reference *exactly* (MAE = 0.000 for every test case), confirming correctness of both the distance computation and the histogram equalization step.

K-Means MAE vs. sequential is 0.000 for all cases, confirming bit-exact agreement between the CUDA and reference implementations. Both use the first k points as initial centroids, integer-truncated centroid updates ($\lfloor \text{sum}/\text{count} \rfloor$), and a lexicographic tie-breaking rule on centroid coordinates—ensuring fully deterministic, reproducible convergence.

6 Discussion

6.1 Summary of Optimisation Gains

Table 11 consolidates the peak speedup observed for each optimisation across all test groups.

Table 11: Summary of optimisation impacts.

Optimisation	Best-case speedup	Regime
Tiled KNN	1.13×	$N = 100$K, $k = 32$
Tiled KNN	$\approx 1\times$	$k = 128$ (register spill dominates)
Spatial Hash Approx	4.5×	$N = 1000$
Spatial Hash Approx	1.1×	$N = 100$K
Approx vs. Exact	6–14×	all N tested

6.2 Register Pressure and Tiling Limits

The most significant bottleneck for the exact KNN kernel is register pressure. Each thread holds `heap_dist[128]` (1 kB), `heap_idx[128]` (512 B), and `hist[256]` (1 kB) – approximately 640 registers if kept live simultaneously. The SM on sm_35 has 65536 registers and limits each block to 65536 registers, so BLOCK_SIZE = 128 and 640 registers/thread exceeds the 512-register/thread limit, causing spilling. Tiling reduces bandwidth pressure but cannot eliminate the register-spill penalty, which is why large-k cases show no tiling benefit.

6.3 Approximate KNN Accuracy

Across all uniform and skewed test cases, MAE is in the range 0.05–0.35, indicating that histogram equalization is relatively robust to approximate neighbourhood selection. The notable exception is clustered data (MAE $\approx$ 0), where the grid structure perfectly mirrors the data structure. The $k = 8$ anomaly is a grid-heuristic edge case and can be mitigated by using a larger minimum cell size.

6.4 Spatial Hashing vs. Grid: When to Use Which

For $N < 10\,000$ the spatial hash provides a meaningful speedup (1.2–4.5×) while maintaining identical MAE. For $N \geq 50\,000$ the gain shrinks to $\sim$ 5–10%, so either variant is acceptable for moderate coordinate ranges. In practice the hash variant is preferable whenever coordinate range is large (avoiding the 10M-cell cap issue in the grid) or when sparse occupancy makes flat-grid traversal expensive.

7 Conclusion

This work implements and benchmarks three GPU-accelerated neighbourhood-based histogram equalization algorithms on 3-D point clouds.

The key findings are:

1. **Exact KNN** correctly matches a sequential reference (MAE = 0) and shows clear $O(N^2)$ scaling. Shared-memory tiling provides up to 13% speedup at $N = 100\,000$, $k = 32$; gains diminish at high k due to register spilling.

2. **Approximate KNN** is 6–14× faster than exact KNN with a typical equalized-intensity MAE of 0.05–0.25. Spatial hashing replaces the flat grid with a compact ($2N$-entry) hash table, giving up to 4.5× speedup at small N and identical accuracy.

3. **K-Means** runtime is nearly independent of T thanks to on-device convergence detection; it is often the fastest method, while Approximate KNN is faster for some small-N settings.

4. Data distribution strongly affects approximate KNN performance: skewed distributions incur 4× slowdown relative to uniform; clustered distributions achieve zero approximation error.

Future work could explore warp-shuffle reductions to eliminate shared memory entirely from the tiling step, or multi-GPU parallelism for $N > 100\,000$.